МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”
ІНСТИТУТ КОМП’ЮТЕРНИХ НАУК ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
кафедра систем штучного інтелекту



ЗВІТ

про виконання лабораторної роботи № 3

з курсу ***«Методологія та процеси моделювання у штучному інтелекті»***

Виконав:
ст. групи КНСШ-13
Дуда Артем Сергійович
Перевірив:
проф. каф. СШІ, д.т.н.,
с.н.с.,
Косаревич Р. Я.

ЛЬВІВ – 2026

Зміст

1. Завдання....3
2. Хід роботи....5
2.1. Набір даних та методика експерименту....5
2.2. Внесення контрольованого шуму....5
2.3. Baseline-модель....7
2.4. Порівняння Grid Search, Random Search і Bayesian Optimization....7
2.5. Дослідження витоку даних....9
2.6. Вкладена крос-валідація....10
3. Висновки....12
4. Список використаних джерел....14

Тема: Оптимізація гіперпараметрів, вкладена крос-валідація та дослідження витоку даних (Data Leakage).

Мета: Навчитися застосовувати методи оптимізації гіперпараметрів та крос-валідації; дослідити вплив вибору алгоритму пошуку на обчислювальну складність і якість моделей; опанувати методологію запобігання витоку даних та застосування вкладеної крос-валідації (Nested Cross-Validation).

1. Завдання

Етап 1. Підготовка даних та створення baseline-моделі

Обрати модель з Лабораторної роботи №2 (з високою кількістю гіперпараметрів або схильну до перенавчання, наприклад: Random Forest, XGBoost, Support Vector Machine, Gradient Boosting).

Зафіксувати метрику якості відповідно до типу задачі (наприклад, F_1-score, ROC-AUC для класифікації або RMSE/MAE для регресії).

Етап 2. Дослідження 1: Порівняння методів оптимізації (Grid vs. Random vs. Bayesian)

Визначити простір гіперпараметрів для обраної моделі (не менше 3–4 параметрів).

Провести оптимізацію трьома методами:

- Full Grid Search (GridSearchCV)
- Random Search (RandomizedSearchCV) з N={20,50} ітераціями
- Bayesian Optimization (використовуючи Optuna або BayesSearchCV) з N = 20 ітераціями

Зафіксувати час виконання кожної стратегії та найкраще значення валідаційної метрики.

Етап 3. Дослідження 2: Оцінка впливу Data Leakage

Побудувати два експериментальних пайплайни:

- Некоректний пайплайн (з витоком)
- Коректний пайплайн (без витоку)

Порівняти отримані валідаційні метрики та оцінити величину штучного завищення якості.

Етап 4. Дослідження 3: Оцінка незміщеної якості (Nested Cross-Validation)

Реалізувати схему Вкладеної крос-валідації:

- Зовнішня петля: K=5 фолдів для оцінки узагальнювальної здатності.
- Внутрішня петля: K=3 фолди для підбору гіперпараметрів.

Порівняти тестову оцінку з внутрішньої CV та незміщену оцінку із зовнішньої CV у Nested CV.

Вимоги до звіту та підсумкового аналізу

У звіт обов'язково мають бути присутні:

- Порівняльна таблиця методів оптимізації:

Метод оптимізації	**Кількість ітерацій / оцінок**	**Час виконання (сек)**	**Best Score (Inner CV)**	**Unbiased Score (Nested CV)**

- Порівняння витоку даних: Таблиця/графік різниці метрик для коректного та некоректного препроцесингу.
- Графік залежності "Якість моделі / Час обчислень".
- Висновки: Обґрунтування доцільності вибору того чи іншого методу оптимізації для реальних задач із врахуванням обмежених обчислювальних ресурсів.

2. Хід роботи

2.1. Набір даних та методика експерименту

Для дослідження використано той самий набір даних, що і в лабораторній роботі 2: bedtime_screentime_sleep_debt.csv (Sleep Debt and Screen Time: Late Night Phone Habits, Kaggle). Після видалення стовпця user_id у таблиці залишається 8500 записів, 16 ознак (12 числових і 4 категоріальні) та цільова змінна sleep_debt_category з чотирма класами. Пропущених значень і дублікатів немає. Класи незбалансовані: Moderate Debt налічує 4462 об'єкти (52.5 відсотка), Mild Deficit 2004 (23.6), Optimal Recovery 1387 (16.3), Severe Sleep Debt 647 (7.6).

Як модель обрано RandomForestClassifier, яка вже використовувалася в лабораторній роботі 2. Вона має багато гіперпараметрів, схильна до перенавчання і водночас швидко навчається на 5695 об'єктах тренувальної вибірки, тому дозволяє порівняти методи оптимізації на достатній кількості конфігурацій за прийнятний час.

Основна метрика якості - f1_macro. Через незбалансованість класів accuracy завищує якість моделі, яка впевнено розпізнає лише найбільший клас, тому використовується макро-усереднений F1, який однаково зважує всі чотири класи. Дані розбито на тренувальну і тестову частини у пропорції 67 на 33 зі стратифікацією за цільовою змінною і фіксованим random_state=42. Тестова частина (2805 об'єктів) не бере участі в підборі гіперпараметрів і використовується лише для фінального контролю.

2.2. Внесення контрольованого шуму

Набір даних [1] синтетичний: цільова змінна майже детерміновано визначається ознаками, тому навіть модель з типовими гіперпараметрами дає F1-macro близько 0.998. За таких умов будь-який метод оптимізації знаходить майже однакові за якістю конфігурації, і порівнювати методи немає сенсу. Щоб

отримати реалістичну задачу, у числові ознаки внесено гауссів шум.

Для цього реалізовано власний трансформер GaussianNoise, який додає до масштабованих числових ознак випадкову величину з нормальним розподілом, математичним сподіванням 0 і стандартним відхиленням scale. Трансформер розміщено всередині ColumnTransformer у числовій гілці після RobustScaler, тому значення scale вимірюється в одиницях міжквартильного розмаху вже масштабованих ознак. Важлива технічна деталь: метод fit трансформера зобов'язаний встановлювати атрибут n_features_in_, інакше scikit-learn 1.9 не вважає конвеєр навченим, коли кастомний трансформер стоїть останнім кроком.

Рівень шуму підібрано емпірично. Для кожного значення scale виконано 3-фолдову крос-валідацію моделі з типовими гіперпараметрами. Отримано такі результати у *таблиці 2.1*.

Таблиця 2.1. Результати додавання шуму

scale	**F1-macro**
0.00	0.9979
0.10	0.9015
0.15	0.8574
0.20	0.8289
0.25	0.7984
0.30	0.7811
0.35	0.7613
0.40	0.7407

Обрано scale = 0.25 (*Рис. 2.1.*). За такого рівня базова модель дає приблизно 0.80, тобто залишається нижче цільової стелі 0.83, але зберігається простір для покращення гіперпараметрами. Шум застосовується всередині конвеєра, тому під час крос-валідації він додається і до тренувальних, і до

валідаційних фолдів, а статистики масштабування обчислюються лише на тренувальній частині. Генератор випадкових чисел зафіксовано, тому експеримент відтворюваний.

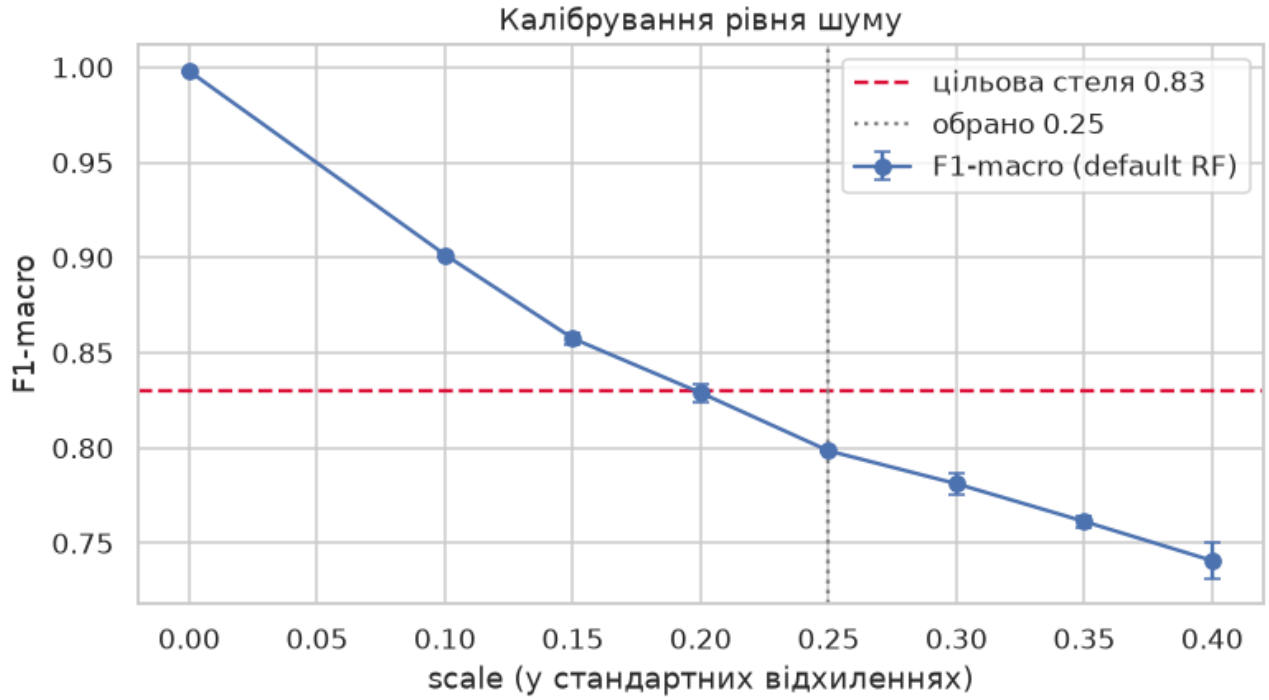


Рисунок 2.1. Калібрування рівня шуму.

2.3. Baseline-модель

Початкова модель використовує типові гіперпараметри: n_estimators=100, max_depth без обмеження, max_features=sqrt, min_samples_leaf=1. Якість оцінено 5-фолдовою стратифікованою крос-валідацією на тренувальній вибірці та окремо на відкладеній тестовій вибірці.

Отримано F1-macro 0.8026 +/- 0.0156 на крос-валідації та 0.8148 на holdout. Час 5-фолдової крос-валідації становить приблизно 1.9 с, тобто модель справді дешева в навчанні. Різниця між крос-валідаційною і holdout-оцінками (близько 0.012) пояснюється розміром тестової вибірки і не є ознакою проблеми.

За класами на holdout отримано такі значення F1: Mild Deficit 0.700, Moderate Debt 0.902, Optimal Recovery 0.798, Severe Sleep Debt 0.859. Найгірше модель розпізнає клас Mild Deficit, який межує з найбільшим класом Moderate

Debt, а найменший клас Severe Sleep Debt розпізнається добре завдяки виразним ознакам (*Рис. 2.2*).

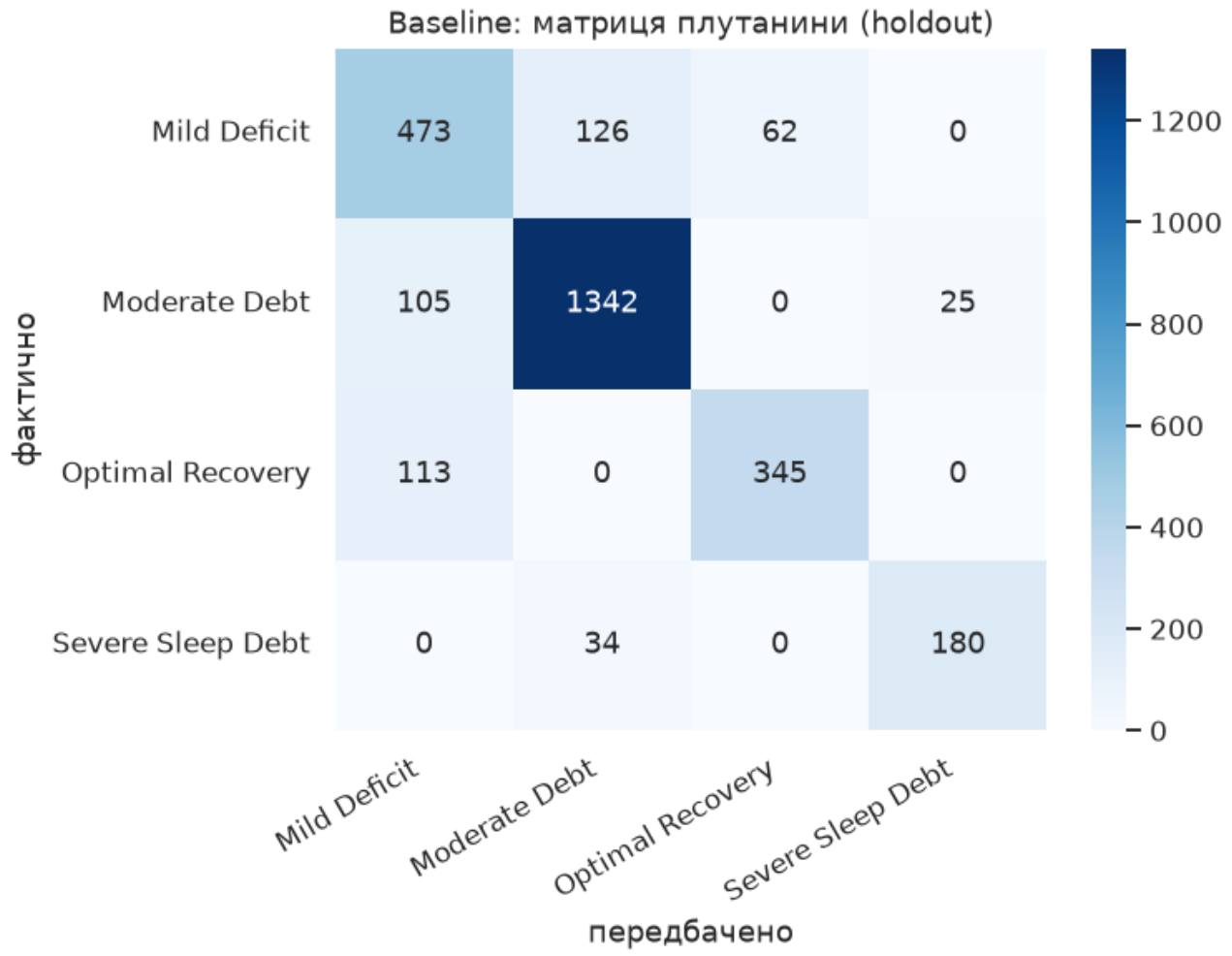


Рис. 2.2. Матриця плутанини baseline-моделі на тестовій вибірці.

2.4. Порівняння Grid Search, Random Search і Bayesian Optimization

Простір пошуку містить чотири гіперпараметри: n_estimators (100, 300, 500), max_depth (10, 20, None), max_features (sqrt, log2, 0.5) і min_samples_leaf (1, 2, 4). Повна сітка налічує 81 конфігурацію. Усі методи використовують однакові фолди StratifiedKFold(3, shuffle=True, random_state=42) і однакову метрику f1_macro, тому різниця в часі відображає саме вартість стратегії пошуку, а не різницю в протоколі оцінювання.

Кількість навчань моделі становить: для GridSearchCV 243 (81 конфігурація на 3 фолди), для Random Search з 20 ітераціями 60, з 50 ітераціями 150, для Optuna з 20 тріалами 60. Як метод байєсівської оптимізації

використано Optuna з семплером TPE, оскільки scikit-learn не має вбудованого байєсівського пошуку, а BayesSearchCV потребує окремої бібліотеки scikit-optimize. Результати порівняння наведено нижче у таблиці 2.2.

Таблиця 2.2. Результати порівняння способів підбору гіперпараметрів

Метод	**N**	**CV F1**	**Holdout**	**Час, с**
GridSearchCV (повна сітка)	81	0.8065	0.8162	143.6
Random Search, N=20	20	0.8041	0.8136	31.5
Random Search, N=50	50	0.8048	0.8165	83.4
Optuna TPE, N=20	20	0.8065	0.8162	127.6

Найкращі конфігурації: GridSearchCV і Optuna обрали n_estimators=500, max_depth=10, max_features=sqrt, min_samples_leaf=1. Random Search з обома бюджетами обрав n_estimators=300, min_samples_leaf=2 і близькі значення інших параметрів, що дало дещо нижчу, але практично ту саму якість.

Повна сітка дала найвищу валідаційну оцінку 0.8065, однак її вартість зростає добутком розмірів простору: додавання п'ятого параметра з трьома значеннями збільшило б сітку до 243 конфігурацій і приблизно втричі збільшило б час. Random Search з 20 ітераціями досяг 0.8041 за 31.5 с, тобто майже 99.7 відсотка якості повної сітки за 22 відсотки часу. Збільшення бюджету до 50 ітерацій додало лише 0.0007 F1-macro за додаткові 51.9 с, що для цієї задачі не виправдано.

Optuna TPE знайшла ту саму конфігурацію, що і повна сітка, витративши 20 оцінок замість 81. Водночас за абсолютним часом вона виявилася повільнішою (127.6 с проти 143.6 с у GridSearchCV лише трохи швидше, а на оцінку витрачала 6.38 с проти 1.77 с). Причина в різній схемі паралелізації: GridSearchCV розподіляє між процесорами окремі конфігурації, а Optuna паралелить фолди всередині одного тріала і додатково витрачає час на службові

операції. Перевага байєсівської оптимізації проявляється тоді, коли одне навчання моделі коштує хвилини або години: тоді критичною стає кількість оцінок, а не ступінь паралелізму. На *Рис. 2.3.* можна побачити час виконання пошуку гіперпараметрів і його результати.

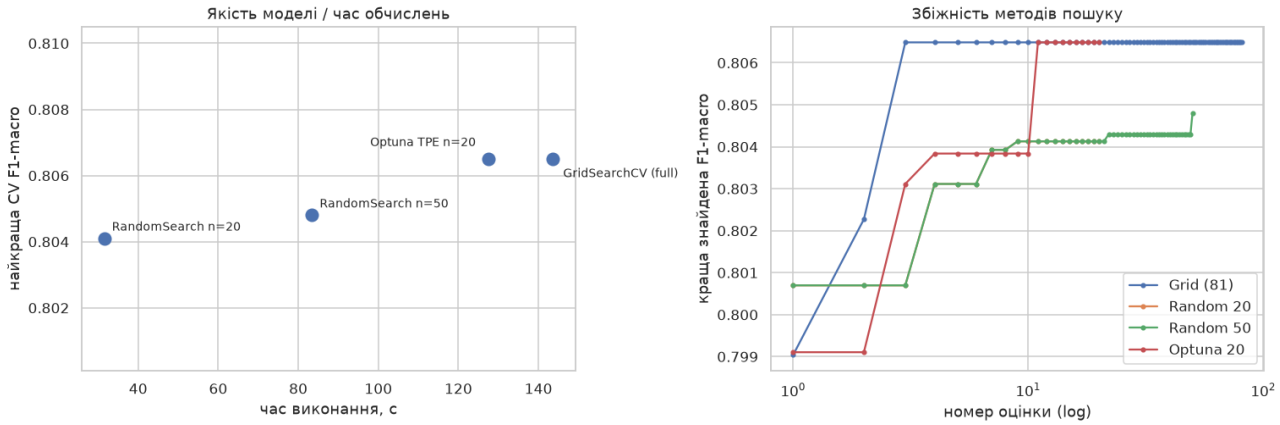


Рисунок 2.3. Співвідношення якості та часу обчислень і збіжність методів оптимізації.

2.5. Дослідження витоку даних

Досліджено три варіанти з однаковою фінальною моделлю RandomForestClassifier(n_estimators=300) і однаковим препроцесором (RobustScaler для числових ознак, one-hot кодування для категоріальних, гауссів шум з scale=0.25).

Перший варіант, коректний, будує весь ланцюжок у imblearn.Pipeline: препроцесор, SMOTE і класифікатор. На кожному фолді крос-валідації SMOTE бачить лише тренувальну частину фолда і не торкається валідаційної. Другий варіант, з витоком, застосовує препроцесор і SMOTE до всього набору з 8500 об'єктів, і лише після цього виконує розбиття на тренувальну і тестову частини. Третій варіант є контролем: препроцесор застосовано до всього набору, але без ресемплінгу, після чого виконано розбиття. Результати наведено у таблиці 2.3.

Таблиця 2.3. Порівняння варіантів пайплайнів.

Варіант	**CV F1-macro**	**Holdout F1**	**Різниця**
Коректний пайплайн	0.8015	0.8129	0.0000
SMOTE до розбиття	0.9095	0.9080	+0.0950
Масштабування до розбиття	0.8098	0.8096	-0.0034

Ресемплінг до розбиття завищує holdout-оцінку на 0.095: модель демонструє 0.908 замість 0.813. Механізм витоку очевидний: SMOTE створює синтетичні об'єкти як інтерполяцію між сусідами одного класу, тому після перемішування частина тестових об'єктів стає майже копіями тренувальних, і модель фактично розпізнає вже бачені точки. У продакшені такої якості не буде.

Масштабування і one-hot кодування до розбиття не дали завищення: різниця становить -0.0034, тобто лежить у межах шуму. Це не витік, оскільки ці кроки не використовують цільову змінну, а випадковий ліс нечутливий до монотонних перетворень числових ознак. Отже, витік створюють не будь-які кроки препроцесингу до розбиття, а саме ті, що залежать від у (ресемплінг, target encoding, відбір ознак, підбір гіперпараметрів), або ті, що створюють дублікати між тренувальною і тестовою вибірками. Результати на рисунку 2.4.

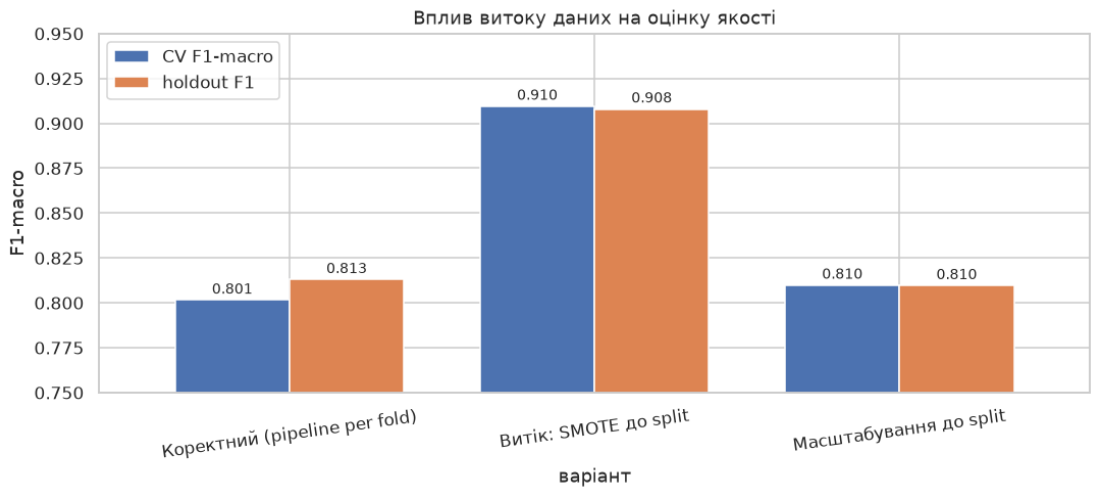


Рис. 2.4. Вплив витоку даних на оцінку якості моделі.

2.6. Вкладена крос-валідація

Оцінка best_score_ з GridSearchCV є оптимістичною, оскільки гіперпараметри обиралися на тих самих фолдах, на яких потім вимірювалася якість. Вкладена крос-валідація розділяє ці дві задачі: зовнішня петля з 5 фолдів оцінює узагальнювальну здатність процедури, а внутрішня петля з 3 фолдів підбирає гіперпараметри всередині кожного зовнішнього тренувального фолда. Процедура виконувалася лише на тренувальній вибірці, тому тестова частина залишилася повністю незайманою. Щоб укластися в прийнятний час, внутрішню сітку скорочено до 24 конфігурацій: n_estimators (100, 300, 500), max_depth (10, None), max_features (sqrt, log2), min_samples_leaf (1, 4). Загалом виконано 5 помножити на 24 помножити на 3, тобто 360 навчань моделі.

Таблиця 2.4. Результати метрики для різних фолдів.

Фолд	**Inner best, зміщена**	**Outer, незміщена**	**Різниця**
1	0.8056	0.8272	-0.0216
2	0.7963	0.7986	-0.0023
3	0.7974	0.7830	+0.0143
4	0.7948	0.8028	-0.0080
5	0.7963	0.7807	+0.0156

Середня внутрішня оцінка становить 0.7981, середня зовнішня 0.7985 +/- 0.0187. Формально середня різниця між внутрішньою і зовнішньою оцінками дорівнює -0.0004, тобто на рівні фолдів оптимізм не проявляється: він повністю тоне в міжфолдовому розкиді, який становить 0.019. Окремі фолди дають різницю від -0.022 до +0.016, причому знак змінюється, що підтверджує випадковий характер коливань.

Водночас систематичне зміщення видно на рівні процедури: best_score_ повної сітки дорівнює 0.8065, а незміщена оцінка зовнішньої петлі 0.7985, тобто завищення становить приблизно 0.008 (*Рис. 2.5.*). Саме тому для

фінального звіту слід брати зовнішню петлю, а не best_score_. Час вкладеної крос-валідації становить 136.8 с.

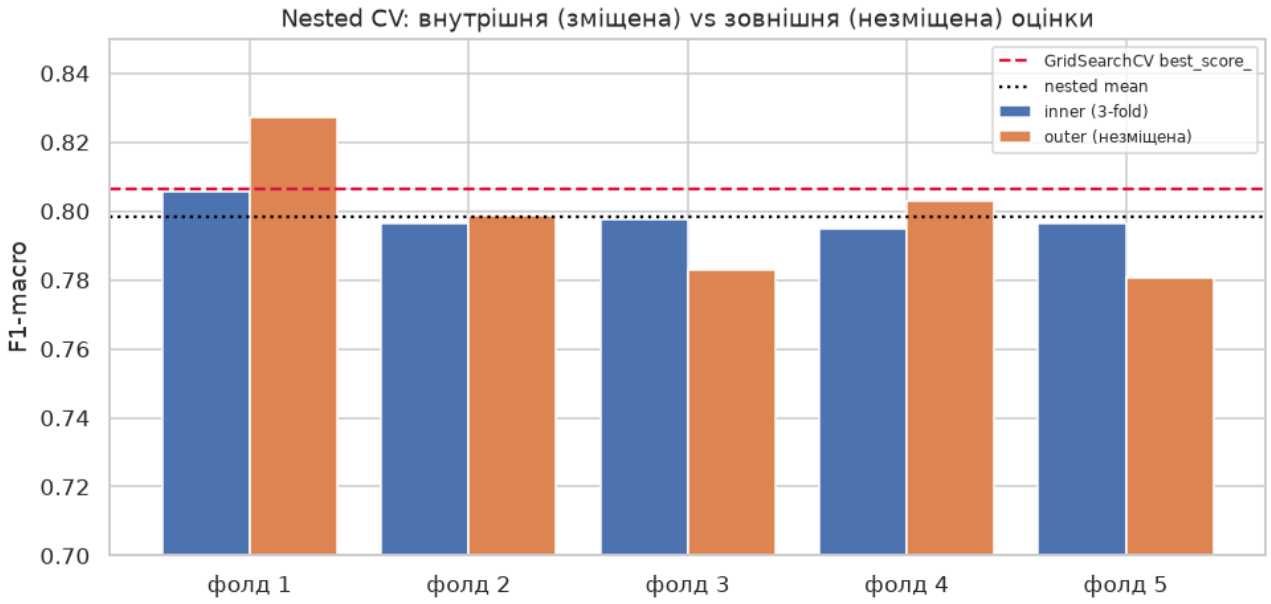


Рис. 2.5. Порівняння внутрішньої і зовнішньої оцінок у вкладеній крос-валідації.

3. Висновки

Щодо методів оптимізації. На цьому наборі повна сітка дала найвищу оцінку 0.8065, але коштувала 143.6 с і 243 навчання моделі. Random Search з 20 ітераціями досяг 0.8041 за 31.5 с, тобто майже 99.7 відсотка якості сітки за 22 відсотки часу. Збільшення бюджету до 50 ітерацій додало лише 0.0007 F1, тому для дешевих моделей достатньо 20-30 ітерацій. Optuna TPE знайшла ту саму конфігурацію, що і повна сітка, за 20 оцінок, але в цьому експерименті програла за абсолютним часом через іншу схему паралелізації. Перевага байєсівської оптимізації зростає, коли одне навчання моделі коштує хвилини або години: тоді важлива кількість оцінок, а не ступінь паралелізму. Практична рекомендація для реальних задач з обмеженими ресурсами: для дешевих моделей і невеликого простору брати Grid Search або Random Search, для дорогих моделей використовувати Optuna з pruning, для проміжних випадків достатньо Random Search на 20-30 ітераціях.

Щодо витоку даних. SMOTE до розбиття завищив holdout-оцінку на 0.095

(0.813 проти 0.908). Причина в тому, що синтетичні об'єкти інтерполюються між сусідами, і частина тестових об'єктів стає майже копіями тренувальних. Масштабування і one-hot кодування до розбиття не дали завищення (різниця -0.0034, у межах шуму), оскільки ці кроки не використовують цільову змінну, а дерева нечутливі до монотонних перетворень ознак. Отже, витік створюють кроки, які залежать від у (ресемплінг, target encoding, відбір ознак, підбір гіперпараметрів) або створюють дублікати між train і test. Їх треба виконувати всередині пайплайна на кожному фолді, а для ресемплінгу використовувати imblearn.Pipeline.

Щодо незміщеної оцінки. best_score_ GridSearchCV (0.8065) є зміщеною оцінкою, оскільки це максимум по 81 конфігурації на тих самих фолдах. Вкладена крос-валідація дала 0.7985 +/- 0.0187, тобто оптимізм best_score_ становить приблизно 0.008. На рівні окремих фолдів різниця між внутрішньою і зовнішньою оцінками коливається від -0.022 до +0.016 і в середньому близька до нуля, тому на 5 фолдах і 5695 об'єктах ефект менший за міжфолдовий розкид. Практичний висновок: різницю моделей меншу за приблизно 0.02 F1 не варто інтерпретувати, вона не відрізняється від шуму фолдів; для фінального звіту слід брати зовнішню петлю, а не best_score_.

Щодо співвідношення факторів. Підбір гіперпараметрів дав 0.004 F1 над baseline (0.8026 проти 0.8065), тоді як внесений шум коштує приблизно 0.20 F1 (0.998 без шуму проти 0.80 з шумом). Тобто на цьому наборі якість обмежена якістю даних, а не гіперпараметрами: жоден пошук не відновить інформацію, якої немає в ознаках. Тому зусилля варто спрямовувати на збір і підготовку даних, а підбір гіперпараметрів розглядати як фінальне доопрацювання.

Обмеження дослідження. Висновки отримано на одному синтетичному наборі даних і одній моделі; абсолютні значення часу залежать від апаратного забезпечення і не переносяться на інші машини. Внутрішня петля містила 3 фолди і скорочену сітку з 24 конфігурацій, тому оцінка незміщеної якості має власну дисперсію. Для остаточних висновків у реальному проєкті варто повторити експеримент на кількох наборах даних і з більшою кількістю фолдів.

4. Список використаних джерел

1. Sleep Debt & Screen Time: Late Night Phone Habits [Електронний ресурс] : датасет / Samar Talwar // Kaggle. — Режим доступу: https://www.kaggle.com/datasets/samartalwar/sleep-debt-and-screen-time-late-night-phone-habits (дата звернення: 11.10.2026).